Systèmes concurrents et distribués

Exercice 1:
La figure 1 représente les différents états dans lesquels un thread Java peut se trouver. Décrivez les états ainsi que les actions qui provoquent le changement d'état d'un thread pour chacune des flèches numérotées.

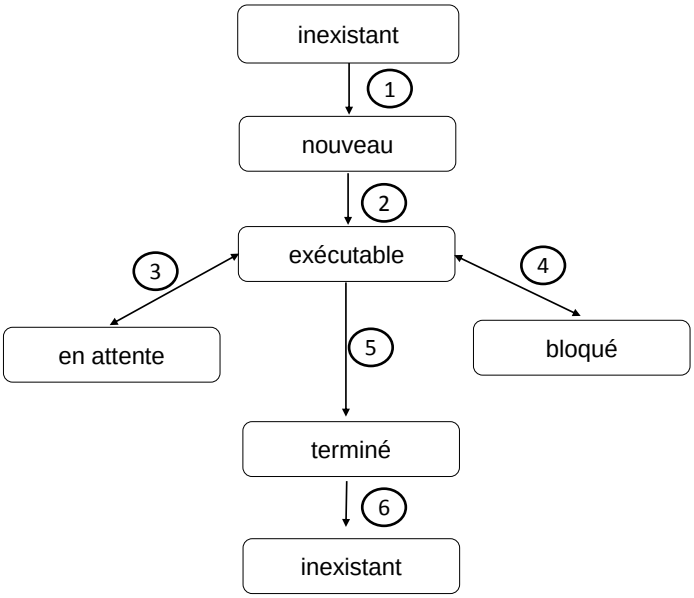

Figure 1: Les différents états d'un thread

Exercice 2:

1. Décrivez le problème du **consensus** et une solution **wait-free** à ce problème.
2. Pour deux processus décrivez ce qu'est l'état du système.
3. Définissez un état **bivalent** et montrez que l'état initial est nécessairement bivalent pour l'exécution d'une solution wait-free au problème du consensus.
4. Définissez un état **critique** et montrez que pour toute exécution d'une solution wait-free au problème du consensus il existe un état critique.
5. Existe-t-il une solution wait-free au problème du consensus, pour deux processus, qui utilise uniquement des registres atomiques (oui/non)?

Exercice 3: Prouvez que l'algorithme de Kessel, voir Figure 2, satisfait l'exclusion mutuelle. Pour cela décrivez les quatre exécutions correspondantes aux orientations possibles des flèches rouges sur la figure (si local[0]=turn[1]→turn[1]=local[1] alors local[0]=y, etc.).

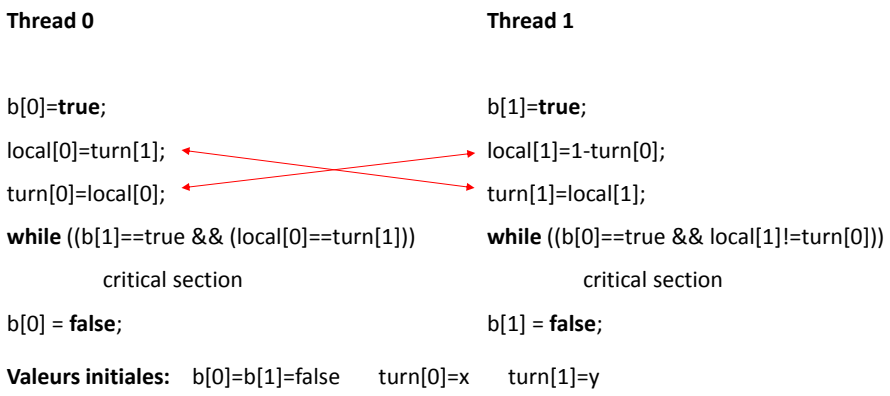

Figure 2: Algorithmes de Kessel

Exercice 4: i) Prouvez que l'algorithme probabiliste solution du problème des philosophes, voir Figure 3, ne peut pas conduire à un inter-blocage.
ii) On numérote les philosophes 0, 1,. . ., n-1 et modifie l'algorithme probabiliste de telle manière qu'un philosophe avec un identificateur impair (pair) choisisse la fourchette de gauche (droite). Montrez que si un philosophe ne mange jamais alors un de ses voisins mange (supposez que l'accès à la fouchette est FIFO).

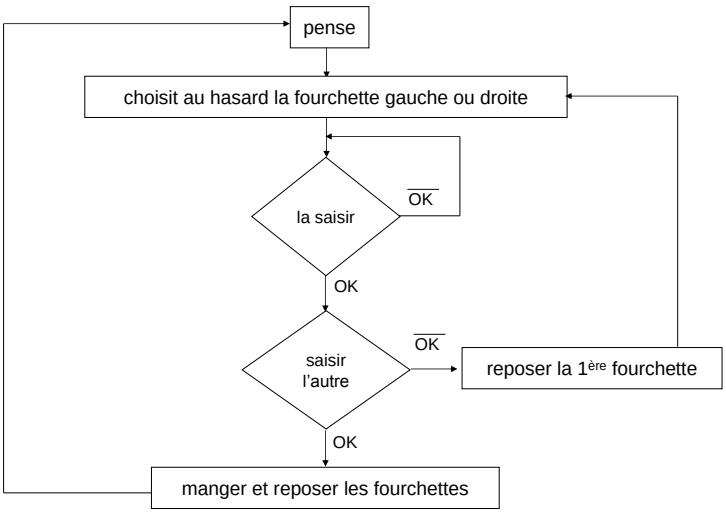

Figure 3: Solution probabiliste du problème des philosophes.

Exercice 5: A la Figure 4 se trouve le code d'une sémaphore vivace. Plusieurs threads qui se partagent une section critique SC, se partagent une sémaphore *LiveSemaphore s* et exécutent s.P() avant SC et s.V() après.

- Montrez que deux threads ne peuvent pas exécuter SC en même temps.
- Montrez que si un thread exécute s.P() alors après un temps fini il exécutera SC.

```
public class LiveSemaphore{
  private volatile int num;
  private Object o;
  public LiveSemaphore() {
    num = 0;
    o = new Object();
  }
  public void P() {
    synchronized(o) {
      if (num > 0)
        try { o.wait();} catch (InterruptedException e) { }
    }
    synchronized(this) {
      num++;
      if (num > 1) try { this.wait();} catch (InterruptedException e) { }
    }
  }
  public synchronized void V() {
    synchronized(o) {
      num−−;
      if (num == 0) o.notifyAll();
      else this.notify();
    }
  }
}
```

Figure 4: Sémaphore équitable

Exercice 6: Implémentez une classe **BoundedBuffer** qui permet d'insérer (deposit) et retirer (fetch) des éléments dans une mémoire tampon de type FIFO. L'implémentation complète le squelette de la Figure 5 et utilise des sémaphores (entières), Figure 6, pour la synchronisation des processus. Le tampon peut-être utilisé par plusieurs producteurs et plusieurs consommateurs.
Pour rappel l'implémentation des sémaphores est rappelée en Figure 6. On ne se préoccupe pas des problèmes de vivacité ou d'équité.

```
class BoundedBuffer {
        final int size = 10; // taille du tampon
        double[] buffer = new double[size]; // tampon
        ....
        public void deposit (double value) {
        ....
        }
        public double fetch() {
        double value;
        ...
        return value;
        }
}
```

Figure 5: Squelette de la classe BoundedBuffer

```
public class CountingSemaphore {
   private int value;
   BinarySemaphore(int initValue) {
      value = initValue;
   }
   public synchronized P() {
      while (value == 0)
      try { this.wait() } catch (InterruptedException e) {}
   value−−;
   }
   public synchronized V() {
      value =++;
      if (value >= 1) notify();
   }
}
```

Figure 6: Implémentation des sémaphores